

# SOC & Blue-Team Operations

The defensive half the lab was missing.

---

Build the operations side of security: how a SOC actually runs — threat intelligence, triage at scale, SOAR automation, EDR, network security monitoring, deception, threat hunting, email authentication, malware sandboxing, and network access control.

```
$ ./deploy --track blue-team --mode operate
```

A direct continuation of *The Complete Lab Manual* (Modules 0–21).

Same lab. Same rules. Everything runs inside your own isolated network.

# How Volume II continues

You finished Volume I with a segmented network, a directory, proxies, a VPN, a SIEM, host and cloud hardening, and a purple-team capstone. You can *build* controls and you can *detect* a handful of attacks. What you do not yet have is the thing employers actually hire for: the ability to *operate* a security function day after day — to run a SOC, reason about threats, investigate at scale, automate the boring parts, and hunt for what your rules miss.

That is this volume. It picks up exactly where Module 21 left off and keeps the same rhythm:

- **Theory** boxes explain why a practice exists in real teams, not in theory.
- **Objective** boxes tell you what you will have running by the end of a task.
- **Dark blocks** are commands you type; **light bordered blocks** are config.
- **Verify** boxes give you a concrete test that it worked.
- **Red Team** boxes attack what you built so detection is earned, not assumed.

## WATCH OUT

The same boundary as Volume I applies, doubled. Every honeypot, beacon simulation, tunnelling test, and emulation in this volume runs against machines *you own* on the *isolated* lab network. Threat-intel feeds and reputation lookups touch the internet read-only; nothing here authorises you to scan, probe, or deceive systems you do not control.

**The through-line.** Volume I gave you *telemetry* (Module 8's SIEM, Module 12's Windows logs, Module 18's ATT&CK map). This volume turns telemetry into *operations*. Each module reuses the lab you already built: the Wazuh/ELK dashboard, the Suricata sensor, the directory, the Kali box, and the attacks you already ran become the raw material for a working SOC.

# Contents

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>How Volume II continues</b>                                      | <b>1</b>  |
| <b>22 Inside the SOC: Roles, Tiers &amp; Workflow</b>               | <b>3</b>  |
| 22.1 Severity, priority, and a triage runbook . . . . .             | 3         |
| 22.2 The metrics that run a SOC . . . . .                           | 4         |
| 22.3 Shift handover and the runbook habit . . . . .                 | 4         |
| <b>23 Threat Intelligence (CTI)</b>                                 | <b>6</b>  |
| 23.1 Stand up MISP and ingest a feed . . . . .                      | 6         |
| 23.2 Enrich an indicator . . . . .                                  | 7         |
| 23.3 Push intel back into detection . . . . .                       | 7         |
| <b>24 Triage &amp; Investigation at Scale</b>                       | <b>9</b>  |
| 24.1 The investigation method . . . . .                             | 9         |
| 24.2 Queries for the attacks you already ran . . . . .              | 9         |
| <b>25 SOAR: Orchestration &amp; Automation</b>                      | <b>11</b> |
| 25.1 Active response: the SOAR you already have . . . . .           | 11        |
| 25.2 A real playbook engine . . . . .                               | 12        |
| <b>26 EDR &amp; Endpoint Telemetry at Depth</b>                     | <b>13</b> |
| 26.1 osquery: the endpoint as a database . . . . .                  | 13        |
| 26.2 Velociraptor: hunt and respond across the fleet . . . . .      | 14        |
| <b>27 Network Security Monitoring: Zeek &amp; PCAP</b>              | <b>15</b> |
| 27.1 Deploy Zeek on the sensor . . . . .                            | 15        |
| 27.2 Hunt 1 — Beaconsing (C2 callback) . . . . .                    | 15        |
| 27.3 Hunt 2 — DNS exfiltration / tunnelling . . . . .               | 16        |
| <b>28 Deception: Honeypots &amp; Canary Tokens</b>                  | <b>17</b> |
| 28.1 Canary tokens . . . . .                                        | 17        |
| 28.2 A honeypot service and a honey account . . . . .               | 17        |
| <b>29 Threat Hunting, Formalized</b>                                | <b>19</b> |
| 29.1 The techniques that find intruders . . . . .                   | 19        |
| 29.2 From hunt to detection . . . . .                               | 20        |
| <b>30 Email Security: SPF, DKIM, DMARC &amp; Phishing Forensics</b> | <b>21</b> |
| 30.1 Publish the records on lab.local . . . . .                     | 21        |
| 30.2 Read what the receiver actually checked . . . . .              | 22        |
| <b>31 Malware Sandboxing &amp; Dynamic Analysis</b>                 | <b>24</b> |
| 31.1 Build the chamber: fake internet + capture . . . . .           | 24        |
| 31.2 Automate it with CAPE . . . . .                                | 25        |
| <b>32 Network Access Control (802.1X &amp; PacketFence)</b>         | <b>27</b> |
| 32.1 Authenticate devices against the directory . . . . .           | 27        |
| <b>33 SOC Skills Checklist (extends Module 21)</b>                  | <b>30</b> |

## MODULE 22

# Inside the SOC: Roles, Tiers & Workflow

A SIEM full of alerts is not a SOC. A SOC is the *process and people* that turn a firehose of signals into decisions: is this real, how bad, who handles it, and what do we do. This module gives you that process, then has you run a single alert from your Module 8 dashboard all the way through it — the loop you will repeat thousands of times in the job.

### THEORY

**What a SOC is, structurally.** A Security Operations Centre is a tiered pipeline. **Tier 1 (triage)** watches the queue, validates alerts, closes false positives, and escalates the real ones — speed and consistency matter more than depth. **Tier 2 (investigation)** takes escalations, scopes them (how many hosts, what timeline), and runs containment. **Tier 3 (hunting / detection engineering / IR)** writes new detections, hunts for what slipped through, and leads serious incidents. Supporting them: a **SOC manager** (metrics, shift coverage) and **detection/automation engineers** (the rules and SOAR from later modules). In small shops one person wears every hat — but the *stages* are always there.

### THEORY

**The alert lifecycle.** Every alert travels the same path: *ingested* (a rule fired) → *triaged* (real or noise?) → *investigated* (scope, severity) → *contained* → *eradicated/recovered* → *closed with a writeup*. Most alerts die at triage as false positives — and that is fine; the skill is closing them fast and correctly without missing the one that matters.

## 22.1 Severity, priority, and a triage runbook

You cannot treat every alert equally. Separate two ideas analysts constantly conflate: *severity* (how bad if true) and *priority* (what to work first = severity × confidence × asset value). A high-severity alert on a throwaway test box is lower priority than a medium one on the directory.

| Priority             | Example signal                           | First action / SLA                    |
|----------------------|------------------------------------------|---------------------------------------|
| <b>P1 Critical</b>   | DC compromise, ransomware, active exfil  | drop everything, page T3, contain now |
| <b>P2 High</b>       | confirmed foothold, brute-force success  | investigate within 30 min             |
| <b>P3 Medium</b>     | single failed-auth burst, IDS scan alert | queue, handle within shift            |
| <b>P4 Low / Info</b> | policy noise, expected admin activity    | batch / tune the rule                 |

### ◎ OBJECTIVE

A documented triage workflow plus a working case-management system on the lab, so that an alert from the Module 8 dashboard becomes a tracked *case* with severity, owner, timeline, and a verdict — not a line that scrolls off a screen.

The open-source stack the industry learns on is **TheHive** (case management) with **Cortex** (analyzer engine for enrichment). Run it on the `soc-workstation` VM from Volume I.

```
# TheHive + Cortex + the data store, via the official docker-compose
git clone https://github.com/StrangeBeeCorp/docker.git thehive-stack
cd thehive-stack/testing      # a single-node lab compose lives here
docker compose up -d         # TheHive on :9000, Cortex on :9001
# first login creates the org + admin; then connect Cortex to TheHive
```

A case needs structure so anyone picking it up mid-incident understands it. The fields you fill on every case:

```
Title:          short, searchable (e.g. "SSH brute force -> proxy-squid
                  10.10.10.20")
Severity:       P1..P4 (see matrix above)
Source:         which sensor fired (Suricata SID, Wazuh rule id, Fail2Ban)
Timeline:       first seen / last seen (UTC -- always UTC)
Scope:          affected hosts, accounts, IPs
ATT&CK:         technique id(s) -- e.g. T1110 Brute Force
Verdict:        true positive / false positive / benign true positive
Actions:        what you did, with timestamps (this is your evidence trail)
```

### VERIFY IT WORKS

Take a live alert from your Module 8 dashboard (re-run the Hydra brute force from Kali if you need one), and open a case in TheHive. Fill every field above. The test is not the tool — it is that you can answer “what happened, how bad, what did you do?” from the case alone, the way a shift handover or an incident ticket demands.

## 22.2 The metrics that run a SOC

### THEORY

**You are measured, so measure yourself.** The numbers every SOC reports: **MTTD** (mean time to detect — attack start to alert), **MTTA** (to acknowledge), **MTTR** (to respond/resolve), **dwelt time** (how long an attacker sat undetected), and the **false-positive rate** per rule. Low MTTD/MTTR with a falling false-positive rate is a healthy SOC. These also tell you where to spend effort: a rule with a 95% false-positive rate is burning analyst hours and must be tuned or retired.

### WATCH OUT

**Metrics get gamed, and alert fatigue is the silent killer.** If you reward “alerts closed per hour,” analysts rush and miss real ones. If a noisy rule fires 400 times a shift, people learn to reflexively close it — and miss the 401st that was real. Treat false-positive rate as a *defect count to drive down*, not background noise to tolerate. Every untuned noisy rule is training your team to ignore the next alert.

## 22.3 Shift handover and the runbook habit

Real SOC run shifts, so unfinished cases must transfer cleanly. A handover note is three lines per open case: *state*, *next action*, *why it matters*. And every recurring alert should have a **runbook**: a short, repeatable “when X fires, check Y and Z, escalate if W.” Writing runbooks for the alerts you

already built (brute force, port scan, web scan from Volume I) is the single highest-leverage thing a new analyst can do — it converts your own knowledge into something the team can execute at 3 a.m.

#### VERIFY IT WORKS

Write one runbook, for the SSH-brute-force alert. Three sections: *what it means*, *steps to confirm* (check `auth.log` for a success after the failures; check Fail2Ban banned the source), *escalate if* (any successful login from the brute-forcing IP). That single page is what turns Module 8's detection into an *operational* control.

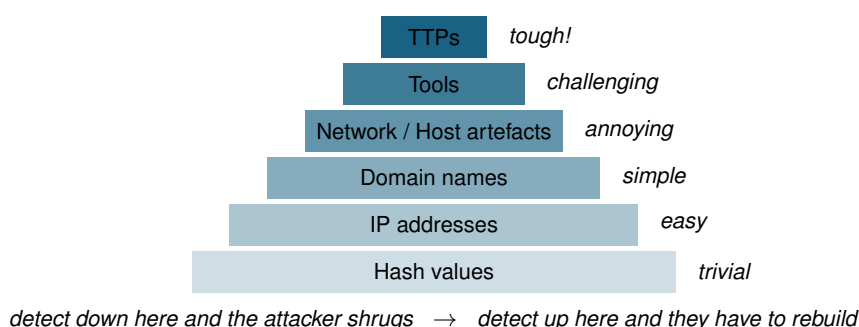
## MODULE 23

# Threat Intelligence (CTI)

Detection answers “did something bad happen here?” Threat intelligence answers “what is bad, who does it, and how?” — so you can detect proactively instead of only reacting. This module stands up an intel platform, ingests real feeds, enriches an indicator, and pushes intelligence back into the SIEM as a watchlist that fires alerts.

### THEORY

**IOCs versus TTPs, and why it matters.** An **Indicator of Compromise** is an artefact of a specific attack: an IP, a domain, a file hash. They are easy to use and easy for the attacker to change. **Tactics, Techniques and Procedures** describe *behaviour* — “uses scheduled tasks for persistence,” “Kerberoasts service accounts” — which an attacker cannot change without changing how they operate. David Bianco’s **Pyramid of Pain** ranks indicators by how much it hurts the attacker when you detect them:



The lesson for your lab: blocking a malicious IP is fine, but it is the bottom of the pyramid — the attacker swaps it in minutes. The detections you wrote in Modules 8 and 18 target *TTPs* (a brute-force *pattern*, a Kerberoast *request type*), which is why they keep working. Intel makes both layers sharper.

### THEORY

**The intelligence cycle and three altitudes.** Intel is produced in a loop: *direction* (what do we need to know?) → *collection* → *processing* → *analysis* → *dissemination* → *feedback*. And it comes at three altitudes: **strategic** (who targets our sector, for leadership), **operational** (which campaigns/groups, for the SOC lead), and **tactical** (the IOCs and TTPs you feed straight into tools). You will work mostly tactical, but knowing the altitude of a piece of intel stops you from blocking a single IP when the real value was the technique behind it.

## 23.1 Stand up MISP and ingest a feed

### ◎ OBJECTIVE

A **MISP** instance on the SOC network ingesting an open threat feed, with indicators that you can pivot on and export straight into your SIEM as a detection watchlist.

```
# MISP via the maintained docker image (lab single-node)
git clone https://github.com/MISP/misp-docker.git && cd misp-docker
cp template.env .env          # set a base URL + admin password for the lab
docker compose up -d          # web UI on https://localhost
# then in the UI: Sync Actions -> Feeds -> enable the default OSINT feeds
# (e.g. the CIRCL OSINT feed, abuse.ch URLhaus / Feodo Tracker)
# and "Fetch and store all feed data" to pull indicators in
```

## THEORY

**Why a platform, not a spreadsheet.** MISP (or OpenCTI) stores indicators *with context*: which event/campaign they belong to, confidence, tags (the **Traffic Light Protocol** for sharing — TLP:RED/AMBER/GREEN/CLEAR), and ATT&CK galaxy tags. That context is what lets you decide *whether to act* on an indicator and *how*, and it lets you correlate a hash you just saw against a known campaign in seconds.

## 23.2 Enrich an indicator

Enrichment turns a bare indicator into a decision. A lone IP means nothing; its reputation, geolocation, passive DNS, and whether it appears on a known-bad list mean everything. In a lab you do this with free, read-only services — the same ones a SOC chains together in an automated playbook (Module 25).

```
# enrich an IOC read-only from the analyst box (these touch the internet, read-
  ↳ only)
whois 185.220.101.1          # ownership / netblock / abuse contact
dig +short 1.2.3.4.in-addr.arpa PTR      # reverse DNS
# free reputation/relationship lookups have public APIs you query with curl:
# abuse.ch (malware/C2), AlienVault OTX, GreyNoise (is it just internet noise?)
curl -s "https://otx.alienvault.com/api/v1/indicators/IPv4/1.2.3.4/general"
```

## RED TEAM — ATTACK

**Why low-pyramid intel decays.** On Kali, stand up a throwaway web listener and note its IP. Add that IP to a MISP event and push it to your SIEM blocklist (next section) — detection fires. Now change the listener's IP (a new lab interface / a new container). The blocklist is instantly blind, even though the *behaviour* (an outbound connection to a brand-new host on a high port) is identical. That is the Pyramid of Pain made concrete: the attacker paid nothing to evade your IOC, which is why you also keep behaviour-based detections.

## 23.3 Push intel back into detection

Intel that sits in MISP catches nothing. The loop closes when indicators become a live watchlist your SIEM matches against. Wazuh and Elastic both consume MISP exports directly.

```
# Wazuh: a CDB list compiled from a MISP export of malicious IPs
# /var/ossec/etc/lists/misp-bad-ips
1.2.3.4:bad
185.220.101.1:bad
# rule that fires when a log event's srcip matches the list
```



```
<rule id="100300" level="12">
  <list field="srcip" lookup="match_key">etc/lists/misp-bad-ips</list>
  <description>Connection involving a MISP-flagged indicator</description>
  <mitre><id>T1071</id></mitre>
</rule>
```

### VERIFY IT WORKS

Generate traffic in the lab from an IP you placed on the list (or replay a log line containing it). The dashboard fires a level-12 alert naming the indicator, and you can pivot from the alert straight to its MISP event for context. You now have an intel-driven detection — the feedback loop, running.

## MODULE 24

# Triage & Investigation at Scale

Watching alerts pop up is monitoring. *Investigating* is taking one signal and answering the five questions that close a case: who, what, when, where, and how far did it spread. This module teaches the query-and-pivot method on the logs you already centralise, using the attacks you already ran as the case material.

### THEORY

**Query languages are the SOC's lingua franca.** Every SIEM has one: **KQL** (Microsoft Sentinel / Defender), **SPL** (Splunk), **Lucene/DSL/EQL** (Elastic), Wazuh's search. They differ in syntax but share three moves: *filter* (narrow to relevant events), *aggregate* (count/group to see shape), and *pivot* (jump from one entity — an IP, a user, a host — to everything related to it). Learn the moves once and you read any of them.

### 24.1 The investigation method

A repeatable method beats cleverness under pressure:

1. **Validate.** Is the alert a true positive? Pull the raw event behind it.
2. **Scope.** Pivot on the key entity (source IP / user / host). How many events, how many hosts, what window?
3. **Timeline.** Order everything by time. First and last seen bound the incident.
4. **Enrich.** Reputation of the IP, role of the user, criticality of the host (Module 23).
5. **Decide.** Severity and priority → contain, escalate, or close with a reason.

### 24.2 Queries for the attacks you already ran

Below, the same investigation expressed in the dialects you will meet. Run the Elastic/Lucene ones against your Module 8 stack; read the others so the syntax is familiar when you switch jobs.

**Brute force (T1110): is there a *success* hiding among the failures?**

```
# Elastic (KQL-style in Kibana Discover) -- failures from one source, then check
  ↳ successes
event.dataset:"system.auth" and event.action:"failed-login" and source.ip
  ↳ : "10.10.0.66"
# Splunk (SPL): shape the attack, then look for the dangerous follow-through
index=auth sourcetype=linux_secure action=failure
| stats count by src_ip, user | sort -count
# the money query: a SUCCESS from an IP that was just failing -> likely cracked
index=auth (action=failure OR action=success) src_ip=10.10.0.66
| transaction src_ip | search action=success
```

**Port scan (T1046): one source fanning out across many ports.**

```
# Splunk: the canonical scan signature -- many distinct dest ports from one src,
  ↳ fast
```

```
index=firewall action=blocked
| stats dc(dest_port) as ports values(dest_port) as portlist by src_ip
| where ports > 50
# Elastic EQL: a sequence of denied connections from the same host
sequence by source.ip with maxspan=30s
[network where event.action == "denied"] [network where event.action == "denied"
  ↳ ""]
```

**Web scan (T1595): a 404 flood from a single client.**

```
# Splunk: directory-brute signature in the nginx access log
index=web sourcetype=nginx:access status=404
| stats count by clientip | where count > 100 | sort -count
```

### VERIFY IT WORKS

Replay the Module 6 chain from Kali (nmap, then Hydra, then gobuster). For each, run the matching query above and answer the five questions in one sentence each: *who* (Kali's IP), *what* (technique), *when* (first/last seen), *where* (which target), *how far* (did it succeed / spread). That paragraph is a complete triage note — exactly what a Tier-1 analyst writes before escalating.

### THEORY

**Pivoting is the whole skill.** A single query rarely closes a case; you *pivot*. Alert says “failed logins from 10.10.0.66” → pivot on that IP across *all* indices (did it also hit the proxy? the web server?) → find it also requested service tickets → pivot on the *account* it used → see that account log in somewhere new. Each pivot widens scope until you have the full picture. Tooling like **Timesketch** makes this visual by laying every source on one timeline.

## MODULE 25

# SOAR: Orchestration & Automation

A human enriching every IP by hand does not scale. SOAR — Security Orchestration, Automation and Response — is the layer that runs the repetitive parts of triage and response as code, leaving humans for judgement. This module builds two playbooks against your lab: one that auto-enriches an alert, and one that auto-contains a high-confidence attack — with the guardrails that stop automation from becoming an outage.

### THEORY

**What SOAR automates, and what it must not.** A **playbook** is a flowchart of actions triggered by an alert: gather context (enrich the IP, look up the user, pull related events), *decide* (rules or a human approval step), then act (block, isolate, open a ticket, notify). Automate the *deterministic, reversible, low-blast-radius* steps — enrichment and ticketing always; containment only with confidence and guardrails. Never fully automate an irreversible or high-impact action (wiping a host, blocking a whole subnet) without a human in the loop.

### 25.1 Active response: the SOAR you already have

Wazuh ships an **active-response** mechanism — the simplest real SOAR. On a detection, the manager tells the agent to run a script. You already saw the manual version (Fail2Ban); this makes it part of the detection pipeline.

```
# /var/ossec/etc/ossec.conf (manager): ban the source of a brute-force rule
<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>    <!-- bundled active-response script -->
  <timeout_allowed>yes</timeout_allowed>
</command>
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100101</rules_id>                <!-- the Module-8 brute-force rule -->
  <timeout>600</timeout>                    <!-- auto-unban after 10 min:
    reversible! -->
</active-response>
```

### WATCH OUT

**The reversible timeout is the guardrail.** An *auto-block with no expiry* is how a SOC accidentally firewalls off its own monitoring server when a rule misfires. A short timeout makes the action self-healing: if it was a false positive, the block lifts itself. Always pair automated containment with a timeout and an allow-list of assets that must *never* be auto-blocked (the DC, the SIEM, the default gateway).

## 25.2 A real playbook engine

For multi-step playbooks (enrich → decide → act → ticket) you want an engine. **Shuffle** is the open-source SOAR the community learns on; **n8n** is a general automation tool that works too.

### ◎ OBJECTIVE

A Shuffle playbook that, when your SIEM raises a brute-force alert, (1) enriches the source IP against a reputation source, (2) opens a TheHive case with that context, and (3) *if* the IP is known-bad *and* not on the protect-list, triggers the reversible block above.

```
# Shuffle (lab single-node)
git clone https://github.com/Shuffle/Shuffle && cd Shuffle
docker compose up -d          # web UI on :3001; build the workflow visually
# the workflow shape (each node is an "app" action):
#   Webhook(alert) -> Enrich(IP reputation) -> Condition(bad? & not protected
#       ↪ ?)
#
#                               |                               |yes-> Block(reversible)
#                               +--> Create TheHive case <-----+
```

The decision node is the part that matters — it is where judgement is encoded:

```
# pseudo-logic of the Condition node (read it as the rules you are encoding)
if reputation.score >= HIGH and src_ip not in PROTECTED_ASSETS:
    action = "auto-contain (reversible 10m) + P2 case"
elif reputation.score >= HIGH:
    action = "P1 case, page on-call -- do NOT auto-block a protected asset"
else:
    action = "P3 case, enrich only, let an analyst decide"
```

### VERIFY IT WORKS

Re-run the Hydra brute force from Kali. Watch the playbook fire end to end: a case appears in TheHive pre-filled with the enrichment, and because Kali's lab IP is not on the protect-list, the reversible block drops it — then lifts itself after the timeout. You have just turned a manual five-minute triage into a five-second automated one, *with* the human-judgement branch preserved for the cases that need it.

### THEORY

**The automation maturity curve.** Teams climb it in order: *manual* (analyst does everything) → *enrichment automated* (context arrives pre-gathered) → *response semi-automated* (analyst clicks “approve block”) → *response automated for high-confidence classes* (with guardrails). Skipping straight to full auto-response is how outages happen. Earn each rung by measuring the false-positive rate of the detection first — the same “tune in detect mode before you block” lesson from Suricata and ModSecurity in Volume I, now applied to response.

## MODULE 26

# EDR & Endpoint Telemetry at Depth

Network logs show traffic; endpoint telemetry shows *behaviour* — what process spawned what, what it touched, what it persisted. Volume I gave you Sysmon on Windows; this module builds the cross-platform, queryable endpoint visibility a modern SOC depends on, then hunts the fleet for a planted persistence mechanism.

### THEORY

**Antivirus vs Sysmon vs EDR.** **Antivirus** matches known-bad files — blind to behaviour. **Sysmon** (Volume I) is rich *logging*, but only logs — it does not respond. **EDR** (Endpoint Detection & Response) adds three things: *behavioural* detection (process *lineage*, not just files), *fleet-wide query* (ask all hosts a question at once), and *response* (isolate a host, kill a process remotely). The open-source way to learn the EDR mindset without a commercial agent is **osquery** (the endpoint as a SQL database) plus **Velociraptor** (fleet hunting and live response).

### 26.1 osquery: the endpoint as a database

#### ◎ OBJECTIVE

osquery deployed on the proxy, web, and client VMs, so you can ask the whole fleet a question — “which hosts have a process listening on an unexpected port?” — and get a table back.

```
# install the agent (Debian/Ubuntu)
sudo apt install osquery -y
# osquery turns OS state into SQL tables you query live:
osqueryi "SELECT name, path, pid FROM processes WHERE on_disk = 0;"    # process
    ↪ whose
                                # binary was deleted -- classic
    ↪ malware
osqueryi "SELECT DISTINCT p.name, l.port FROM listening_ports l
    JOIN processes p ON l.pid = p.pid WHERE l.port NOT IN (22,80,443,3128)
    ↪ ;"
osqueryi "SELECT * FROM crontab;"                                     # persistence via cron
osqueryi "SELECT name, path FROM startup_items;"                    # autoruns
```

A **query pack** runs a set of these on a schedule and ships results to your SIEM — continuous endpoint telemetry, not one-off questions:

```
# /etc/osquery/osquery.conf -- schedule security-relevant queries
{
  "schedule": {
    "rogue_listeners": {
      "query": "SELECT p.name, l.port, l.address FROM listening_ports l
        JOIN processes p ON l.pid=p.pid;",
      "interval": 300
    },
  },
}
```

```

"suid_binaries": {
  "query": "SELECT path, permissions FROM suid_bin;",
  "interval": 3600
},
"new_cron": { "query": "SELECT * FROM crontab;", "interval": 600 }
}

```

## 26.2 Velociraptor: hunt and respond across the fleet

### THEORY

**Why Velociraptor.** osquery answers “what is true now?”; Velociraptor adds *hunts* (run a query across every endpoint and collect results), *artefacts* (pre-built DFIR collections — browser history, shell history, persistence locations), and *live response* (pull a file, kill a process, isolate a host). It is the closest free thing to commercial EDR and is used in real incident response.

```

# Velociraptor: one binary is server + client
velociraptor config generate > server.config.yaml
velociraptor --config server.config.yaml frontend &      # GUI + agent enrolment
# then in the GUI: launch a HUNT across the fleet, e.g. the artefact
#   Linux.Sys.Crontab or Generic.Detection.HuntForProcess
# results stream back as a table you triage like alerts

```

### RED TEAM — ATTACK

**Plant persistence, then catch it fleet-wide.** On the `web-dmz` box, simulate a foothold’s persistence (your own host, lab only): add a benign cron entry and a rogue listener.

```

# on web-dmz (simulating what an attacker would drop)
(crontab -l 2>/dev/null; echo "*/5 * * * * /tmp/.beacon.sh") | crontab -
nohup nc -lvnp 4444 >/dev/null 2>&1 &      # rogue listener on an odd port

```

### VERIFY IT WORKS

From the SOC box, run a fleet hunt for new cron entries and for listeners on non-standard ports. `web-dmz` stands out: the `/tmp/.beacon.sh` cron and the port-4444 listener appear in the results table while the other hosts are clean. You found a foothold by *asking the whole fleet one question* — the core EDR/hunting move. Then practise response: use Velociraptor to kill the listener and collect the cron artefact as evidence before you remove it.

## MODULE 27

# Network Security Monitoring: Zeek & PCAP

Suricata (Volume I) tells you when traffic matches a *signature*. Zeek tells you what every connection was, signature or not — a structured record of who talked to whom, for how long, over what protocol. That metadata is where you catch the attacks no signature exists for yet: beaconing command-and-control and DNS exfiltration. This module adds Zeek to your sensor and hunts both.

### THEORY

**Zeek vs Suricata: behaviour vs signature.** They are complementary, and mature SOCs run both. **Suricata** is a signature/IDS engine — great at “this packet matches a known exploit.” **Zeek** is a network *analysis* framework that turns traffic into rich logs: `conn.log` (every connection, with duration and bytes), `dns.log`, `http.log`, `ssl.log` (with JA3/JA4 TLS fingerprints), `x509.log`, and more. You hunt over those logs for *behaviour*: a connection that repeats with suspicious regularity, an enormous volume of tiny DNS queries, a TLS fingerprint that matches a known malware family. That is Network Security Monitoring.

## 27.1 Deploy Zeek on the sensor

### 🕒 OBJECTIVE

Zeek running on `ids-sensor` alongside Suricata, producing `conn/dns/http/ssl` logs from the same mirrored traffic, shipped into your Module 8 dashboard.

```
sudo apt install zeek -y      # or build from source; package name may be zeek-lts
# point Zeek at the monitored interface (the SPAN/tap from Volume I's topology)
echo "interface=eth0" | sudo tee -a /opt/zeek/etc/node.cfg
sudo /opt/zeek/bin/zeekctl deploy
# logs land here, one structured file per protocol:
ls /opt/zeek/logs/current/    # conn.log dns.log http.log ssl.log x509.log ...
```

## 27.2 Hunt 1 — Beaconing (C2 callback)

### THEORY

**Why beaconing stands out.** Implanted malware “phones home” on a timer — every 30 or 60 seconds, often with jitter. Humans browse irregularly; a beacon is suspiciously *regular*. So you do not need a signature: you look in `conn.log` for the same source-to-destination pair recurring at near-constant intervals with small, similar payloads. **RITA** (Real Intelligence Threat Analytics) automates exactly this scoring over Zeek logs.

### RED TEAM — ATTACK

**Simulate a beacon (lab only).** On Kali, run a fake implant that calls a lab listener on a fixed cadence; on `web-dmz` run the listener. Nothing malicious — it is `curl` on a timer — but it



has the exact *shape* of C2.

```
# web-dmz: a trivial "C2" listener
while true; do printf 'HTTP/1.1 200 OK\r\n\r\n' | nc -lvnp 8443 -q1; done
# Kali: beacon every 60s with a little jitter, like a real implant
while true; do curl -s http://10.10.30.10:8443/ >/dev/null; sleep $((55+RANDOM
    ↪ %10)); done
```

```
# Hunt in Zeek's conn.log: group by src/dst, look for many connections,
# steady interval, small bytes -- the beacon signature.
cat /opt/zeek/logs/current/conn.log \
| zeek-cut id.orig_h id.resp_h id.resp_p duration orig_bytes \
| sort | awk '{c[$1] -> "$2":"$3}++} END{for(k in c) if(c[k]>10) print c[k], k}'
# or let RITA score it automatically:
rita import /opt/zeek/logs/current/conn.log lab && rita show-beacons lab
```

### VERIFY IT WORKS

RITA (or your `conn.log` count) flags the Kali→web-dmz pair with a high beacon score: many connections, near-constant timing, tiny consistent payloads. No signature existed — you caught it on *behaviour*. That is the detection class that finds real C2.

## 27.3 Hunt 2 — DNS exfiltration / tunnelling

### THEORY

**DNS is a covert channel.** DNS is almost always allowed outbound, so attackers smuggle data inside DNS queries (long, high-entropy subdomains) or tunnel a whole session over it. The tells in `dns.log`: an unusual *volume* of queries to one domain, abnormally *long* query names, high *entropy* in the labels, and lots of TXT/NULL record types.

```
# Zeek dns.log hunt: domains receiving an abnormal number of unique long
    ↪ subdomains
cat /opt/zeek/logs/current/dns.log | zeek-cut query \
| awk '{n=split($1,a,"."); print a[n-1]".a[n], length($1)}' \
| awk '{cnt[$1]++; if($2>40) longc[$1]++;} END{for(d in cnt) printf "%s q=%d long
    ↪ =%d\n",d,cnt[d],longc[d]}' \
| sort -k3 -t= -rn | head
```

### VERIFY IT WORKS

Generate a burst of long-label lookups to a single lab-controlled zone (e.g. repeated `dig <random-base32>.exfil.lab.local`). The hunt ranks that zone to the top by query count and long-name count — the DNS-tunnelling signature. You now detect two of the most common evasive channels using metadata alone.

## MODULE 28

# Deception: Honeypots & Canary Tokens

Every detection so far has a false-positive problem, because legitimate activity looks similar to attacks. Deception inverts that: a tripwire nobody has any legitimate reason to touch. When it fires, it is *real* — the highest-fidelity signal in the SOC. This module plants three: a canary token, a honeypot service, and a honey account in your directory.

### THEORY

**Why deception is high-fidelity.** A failed login *might* be a typo. A scan *might* be a misconfigured tool. But a credential that exists only to be stolen, a file that exists only to be opened by an intruder, an account that no human ever logs into — touching any of those has *no innocent explanation*. So a deception alert needs almost no triage: its base false-positive rate is near zero. The cost is setup, not noise. **Low-interaction** honeypots fake a service to catch contact; **high-interaction** ones are real systems you watch closely; **honeytokens** are fake secrets (credentials, files, API keys) that alert on use.

### 28.1 Canary tokens

#### 🕒 OBJECTIVE

A handful of canary tokens scattered where an attacker would look — a fake `aws_keys.txt` in a home directory, a “passwords” document on a share — each firing an alert the instant it is opened or used.

Canarytokens are free and self-hostable; the concept is a unique URL/credential embedded in bait. When the bait is touched, the unique token phones home and you get an alert.

```
# self-host the open-source Canarytokens (lab)
git clone https://github.com/thinkst/canarytokens-docker.git && cd canarytokens-
↳ docker
cp switchboard.env.dist switchboard.env && cp frontend.env.dist frontend.env
docker compose up -d          # generate tokens in the web UI:
#   - a "Word document" token -> drop as ~/Documents/salaries-2025.docx on the
↳ client
#   - an "AWS keys" token      -> drop as ~/.aws/credentials bait on web-dmz
#   - a "DNS/URL" token       -> embed in a fake internal wiki link
```

### 28.2 A honeypot service and a honey account

```
# OpenCanary: a low-interaction honeypot that fakes services and alerts on
↳ contact
pip install opencanary --break-system-packages
opencanaryd --copyconfig          # edit ~/.opencanary.conf: enable fake ssh/ftp/
↳ http
opencanaryd --start              # any connection to the fake service = alert
```

In the directory from Volume I, create a **honey account**: a domain user that looks privileged (an enticing name, a description hinting at admin rights) but is used by nobody. Any authentication attempt against it is malicious by definition.

```
# FreeIPA / AD: a tempting decoy account that no human ever uses
ipa user-add svc-backup-admin --first=Backup --last=Admin \
    --shell=/sbin/nologin      # looks juicy, logs in nowhere
# Then alert on ANY auth involving it:
#   Windows  -> Security Event 4625/4768/4769 where TargetUserName = svc-backup-
    admin
#   FreeIPA   -> a Wazuh rule matching that principal in the KDC log
```

### RED TEAM — ATTACK

**Walk into the trap.** From Kali, replay the Module 1 directory enumeration and the Module 6 service probing. The enumeration surfaces `svc-backup-admin` (it is meant to look like a prize); attempting to authenticate to it, or connecting to the OpenCanary fake SSH, or opening the bait document — any of these trips a tripwire.

### VERIFY IT WORKS

Each tripwire produces an *immediate, P1-grade* alert with essentially zero false positives: the canary document phones home with the opener's source IP; OpenCanary logs the connection; the honey account's auth attempt raises a Windows 4768/4769 (or FreeIPA KDC) event your rule tags critical. Compare the triage effort to Module 24's brute-force investigation — a deception alert needs almost none, because there is no benign explanation. That asymmetry is why every mature SOC plants tripwires.

## MODULE 29

# Threat Hunting, Formalized

Detections catch what you already thought to look for. Hunting is the disciplined search for what they miss — and every find becomes tomorrow's detection. Module 18 introduced the idea; this module gives you a repeatable hunt loop, the techniques that actually surface intruders, and the habit of turning a hunt into a portable detection.

### THEORY

**The hunt loop (PEAK / TaHiTI).** Hunting is hypothesis-driven, not random log-staring. The loop: *form a hypothesis* grounded in a real TTP (“if an attacker were living off the land, I’d expect unusual parent→child process trees”); *gather* the relevant data; *analyze* for the pattern; *conclude* (found something / clean / inconclusive); and crucially *operationalize* — a real find becomes a new detection, an inconclusive hunt becomes a note to collect better data. Three hunt flavours: **intel-driven** (a TTP from Module 23), **hypothesis-driven** (your reasoning), and **baseline/anomaly** (what is rare here?).

### 29.1 The techniques that find intruders

### THEORY

**Stack counting and rarity.** The workhorse technique: aggregate a field across the fleet and look at the *rare* values. A binary seen on one host once, a parent process that should never spawn a shell, an account that logs in interactively at 3 a.m. — attackers hide in the long tail of normal. **LOLBins** (living-off-the-land binaries: `certutil`, `bitsadmin`, `powershell`, `wget` in odd places) are legitimate tools used maliciously, so you hunt their *context*, not their presence.

### ◎ OBJECTIVE

Three structured hunts against your aggregated logs — beaconing, anomalous process lineage, and off-hours privileged logon — each documented and at least one converted into a Sigma rule (Module 8/18).

```
# Hunt A -- rare process: binaries seen on exactly one host, exactly once
# (stack-count process names across the fleet, keep count==1)
index=edr event=process_create | stats dc(host) as hosts count by process_name
| where hosts==1 and count<=2
# Hunt B -- impossible lineage: a web/proxy service spawning a shell
index=edr event=process_create parent_process IN ("nginx","squid","httpd")
process_name IN ("bash","sh","dash","nc","python*")
# Hunt C -- off-hours privileged logon (interactive admin outside business hours)
index=auth action=success user IN (admin, root) | eval h=strftime(_time,"%H")
| where h<6 OR h>21
```

## 29.2 From hunt to detection

### THEORY

**A hunt that finds something but produces no detection was half-finished.** The output of a successful hunt is a *repeatable* rule so you never hunt the same thing twice. Write it as **Sigma** (Module 8) so it converts to Wazuh, Splunk, or Elastic, store it in git with its ATT&CK tag and a test case, and validate it the way Module 18 taught — with Atomic Red Team or a manual replay. Hunting and detection engineering are the same loop at different speeds.

```
# sigma rule born from Hunt B (service process spawns a shell) -- portable
  detection
title: Web/Proxy Service Spawns Interactive Shell
status: experimental
logsource: { category: process_creation }
detection:
  selection:
    ParentImage|endswith: ['/nginx','/squid','/httpd']
    Image|endswith: ['/bash','/sh','/dash','/nc','/python3']
  condition: selection
level: high
tags: [attack.execution, attack.t1059]
```

### VERIFY IT WORKS

Trigger Hunt B's condition on a lab host (have `nginx` spawn a shell via a benign test, or reuse the Module 13 container-shell drill). Convert the Sigma rule with `sigma convert` for your backend, load it, and confirm the replay now fires it automatically. You found something by hunting, then made sure you would *detect* it forever after. Document the three hunts (hypothesis, data, finding, outcome) — that hunt log is a portfolio piece and the raw material for your ATT&CK coverage map.

## MODULE 30

# Email Security: SPF, DKIM, DMARC & Phishing Forensics

The lab can detect a scan, a brute force, a beacon — yet it never looked at the door most attackers actually walk through. Phishing is the number-one initial-access vector in real incidents, and email's original sin is that the sender is trivial to forge. This module publishes the three sender-authentication records on your lab domain, then has you spoof a message and investigate the headers exactly as a SOC analyst triages a reported phish.

### THEORY

**Why email is trivially spoofable, and the three protocols that fix it.** Plain SMTP has no sender verification: the *From* you see is just text the sender typed. Three layered DNS-published controls add trust. **SPF** (Sender Policy Framework) lists which IPs may send for a domain; the receiver checks it against the *envelope* sender (Return-Path). **DKIM** (DomainKeys Identified Mail) has the sending server cryptographically *sign* the message; the public key lives in DNS, so the receiver verifies the mail was authorised and untampered. **DMARC** ties SPF and DKIM to the *visible* From domain through *alignment*, tells receivers what to do on failure (none / quarantine / reject), and requests reports. Alignment is the crux: SPF or DKIM can pass for *some* domain, but DMARC demands that the authenticated domain match the From the human reads.

## 30.1 Publish the records on lab.local

### ◎ OBJECTIVE

Your lab mail domain (a small Postfix on the directory host, or any lab MTA) publishes valid SPF, DKIM and DMARC records, starting in monitor mode and progressing to enforcement — so a forged internal email is provably rejected while legitimate mail passes.

```
; lab.local zone -- the three sender-authentication records
lab.local.                IN TXT "v=spf1 mx ip4:10.10.10.5 -all"    ; only the
    MX may send
default._domainkey.lab.local. IN TXT "v=DKIM1; k=rsa; p=MIGfMA0G...<public-key>"
_dmarc.lab.local.          IN TXT "v=DMARC1; p=none; rua=mailto:dmarc@lab.
    local; fo=1"
;   roll the DMARC policy forward only as confidence grows:
;   p=none (monitor + collect reports) -> p=quarantine -> p=reject
```

```
# generate the DKIM keypair; publish the public half as the TXT record above
sudo apt install opendkim opendkim-tools postfix -y
opendkim-genkey -s default -d lab.local          # writes default.private + default.
    ↪ txt
# wire OpenDKIM into Postfix as a milter so outbound mail is signed:
#   /etc/postfix/main.cf -> smtpd_milters = inet:localhost:8891
sudo systemctl restart opendkim postfix
```

## 30.2 Read what the receiver actually checked

### THEORY

**The headers tell the whole story.** A receiving server records its verdict in the **Authentication-Results** header (`spf=pass, dkim=pass, dmarc=fail, etc.`). Two more tells matter: a mismatch between the *Return-Path* (envelope sender) and the visible *From* is the classic spoof signature, and the **Received** chain — read bottom-up, oldest hop first — shows where the message truly originated. A phish often shows an originating IP that has no business sending for the claimed domain.

```
# verify any domain's published policy yourself (read-only DNS lookups):
dig +short TXT lab.local                # SPF record
dig +short TXT default._domainkey.lab.local # DKIM public key
dig +short TXT _dmarc.lab.local          # DMARC policy + reporting address
# in a suspect message, the lines a SOC analyst reads first:
# Authentication-Results: ... spf=? dkim=? dmarc=?
# Return-Path / envelope-from vs From:      <- mismatch = likely spoof
# Received: (walk the chain bottom-up to find the true origin IP)
```

### RED TEAM — ATTACK

**Forge an internal sender (your own mail server only).** From Kali, send a message that *claims* to be from a trusted internal address to a lab mailbox. With DMARC in `p=none` it lands; with `p=reject` the receiver refuses it.

```
# swaks -- the Swiss-army SMTP tester; lab MTA only
swaks --to victim@lab.local --from ceo@lab.local --server 10.10.10.5 \
  --header "Subject: Urgent wire transfer" \
  --body "Please action this transfer immediately."
# Kali's IP is not in SPF and the mail is not DKIM-signed for lab.local
# -> dmarc=fail -> rejected once you enforce. That is the spoof, blocked.
```

### VERIFY IT WORKS

Inspect the delivered (or bounced) message's headers. The spoof produces `spf=fail` (or `softfail`) and `dkim=none`, so DMARC fails on alignment — and with `p=reject` the receiver drops it while genuine internal mail still passes. The test of understanding: point to the exact `dmarc=fail` line and explain, in one sentence, *why* alignment failed. Then practise the real job — take a phishing email you actually received, analyse its headers, and produce a verdict (spoofed domain? lookalike? compromised account?) from the headers alone.

### WATCH OUT

**What these protocols do not stop, and how to roll them out.** SPF breaks on plain forwarding and mailing lists (the path changes); DKIM survives unless the body is altered — which is exactly why **ARC** exists and why you start at `p=none`, read the aggregate reports, then move to quarantine and finally reject, the same tune-before-enforce discipline as Suricata and ModSecurity. And remember the gap: SPF/DKIM/DMARC stop *domain* spoofing, not *lookalike* domains (`ceo@lab.local`) or mail from a genuinely *compromised* account. Those need user

reporting, the sandbox (next module) for attachments, and the intel pipeline (Module 23).

**Pipeline tie-in.** A reported phish becomes a SOC case (Module 22); its sender IP, URLs and attachment hashes become indicators in MISP (Module 23); and any attachment goes straight to the sandbox you build next.



# Malware Sandboxing & Dynamic Analysis

You can now detect an attack and read a phishing header — but you cannot yet answer the question every triage eventually reaches: “*what does this file actually do?*” A sandbox detonates a suspicious sample in isolation and records its behaviour. This module builds one safely and uses it on a deliberately harmless specimen, so you learn the method without ever handling live malware.

## THEORY

**Static versus dynamic analysis.** **Static** analysis examines a file without running it: hashes, embedded strings, PE/ELF structure, YARA signatures. **Dynamic** analysis *runs* the sample in an instrumented, isolated environment and watches what it does: processes spawned, files and registry keys written, and — most valuable — network callbacks. A sandbox automates the dynamic side and produces the artefacts you actually report: a process tree, file and network IOCs, and a behavioural verdict mapped to ATT&CK.

## WATCH OUT

**Isolation is the entire safety model — read this before anything else.** A detonation chamber that can reach your network or the internet is a weapon, not a tool. The rules: a *dedicated* host or a fully isolated VLAN with **no route** to your real network or the internet; *VM snapshots* that you revert after every single detonation; the assumption that a sample may detect a VM and try to escape, so you never detonate on a machine that touches production or the lab’s trusted zones; and a *fake internet* so the sample reveals its callbacks without truly reaching out. Finally: only ever handle samples you are authorised to. This module uses a **benign** specimen, never live malware.

## 31.1 Build the chamber: fake internet + capture

### ◎ OBJECTIVE

An isolated analysis VM wired to a gateway running **INetSim** (which fakes DNS, HTTP, SMTP and more), with full packet capture and behavioural monitoring, so a detonated specimen reveals its network and host behaviour while reaching nothing real.

```
# on the isolated gateway VM: INetSim answers every service so the sample
# "phones home" -- to your fake internet, where you record it.
sudo apt install inetsim -y
# /etc/inetsim/inetsim.conf -> enable dns/http/https/smtp, bind to the lab-only
#   ↪ IP
sudo systemctl start inetsim
# on the analysis VM, capture EVERYTHING for the detonation window:
sudo tcpdump -ni any -w /evidence/detonation.pcap &
# (Windows analysis VM: add Sysmon + Procmon; Linux: add auditd/Falco from Vol I)
```

A **benign** specimen lets you see exactly what the telemetry should look like before you ever face the real thing. It is harmless — it only produces observable signals, the same idea as the Module 27 beacon simulation:

```
#!/bin/sh
# ---- BENIGN analysis specimen (NOT malware) -- produces observable telemetry
----
nslookup updates.evil-c2.test           # DNS lookup -> captured by INetSim
curl -s http://updates.evil-c2.test/gate # fake C2 callback -> captured
touch "$HOME/.cache/marker_dropped"     # a "dropped file" artefact
nc -lnp 4444 &                          # opens a listener -> observable port
```

## 31.2 Automate it with CAPE

### THEORY

**Why an automated sandbox.** Doing the above by hand teaches the moving parts; at volume you want a sandbox that orchestrates the snapshot, detonation, capture and report for you. **CAPE** is the maintained, modern successor to Cuckoo (interactive cloud services such as Any.Run and Joe Sandbox fill the same role commercially). The report is the deliverable: process lineage, dropped files, contacted domains/IPs, and an ATT&CK mapping you can act on.

```
# CAPE -- orchestrates VM snapshots, detonation, capture and reporting
git clone https://github.com/kevoreilly/CAPEv2.git
# follow CAPE's installer (it manages the isolated analysis guest + INetSim/
  ↳ fakenet)
# submit a sample, then read the structured report:
cape submit specimen.sh           # -> process tree, network IOCs, dropped files,
  ↳ ATT&CK
```

For the *detection* half, use **EICAR** — the industry-standard *harmless* antivirus test file. Download it from [eicar.org](http://eicar.org); do *not* paste its string into shared documents, because scanners will quarantine the document itself. Dropping it on a monitored host should trip your AV and Suricata, proving the alert path end to end.

### VERIFY IT WORKS

Detonate the benign specimen. Your `detonation.pcap` shows the DNS lookup and the HTTP request to the fake C2 (answered by INetSim), the dropped `marker_dropped` file appears on disk, and the port-4444 listener is visible — and CAPE lists all of these as IOCs with an ATT&CK mapping. You have extracted behavioural indicators from a sample you never let touch anything real. Push those IOCs to MISP (Module 23) and write a detection for them (Module 18). With EICAR, confirm the AV/IDS path lights up. That is the full loop: *detonate* → *observe* → *extract IOCs* → *detect*.

### THEORY

**Why analysis is an arms race.** Real malware fights back: it checks for VM artefacts, sleeps to outlast a short detonation window, or waits for user interaction before acting. That is why dynamic analysis is paired with static analysis, why detonation windows and interactive sandboxes exist, and why a clean sandbox run is evidence of nothing on its own. Knowing the limits keeps you honest about a “benign” verdict.

**Pipeline tie-in.** The malicious attachment from the Module 30 phish is exactly what lands here:

*phishing report (M30) → detonate (M31) → IOCs to MISP (M23) → detections and hunt (M18, M29) → block and incident response (M19).*

## MODULE 32

# Network Access Control (802.1X & Packet-Fence)

Your firewall decides what traffic may cross *between* segments. But nothing yet decides *who is allowed onto the network at all* — plug into a LAN port or join the Wi-Fi and you get an IP on the trusted network. Network Access Control closes that door: a device must prove who it is before the port even opens. This module authenticates devices with 802.1X against the directory you built in Module 1 and the PKI from Module 10.

### THEORY

**The NAC problem and the 802.1X cast.** Port-based access control puts authentication *before* connectivity. Three roles: the **supplicant** (the device), the **authenticator** (the switch or access point), and the **authentication server** (RADIUS). The switch port passes only EAPOL frames until RADIUS approves the device, then drops it into an assigned VLAN. EAP methods set the strength: **EAP-TLS** is certificate-based (the strongest — it reuses your Module 10 CA), while **PEAP / EAP-TTLS** authenticate credentials against the directory (Module 1). For devices that cannot speak 802.1X — printers, IoT — there is **MAB** (MAC Authentication Bypass), which is weaker and spoofable. NAC can also *profile* device type and check posture before granting access.

## 32.1 Authenticate devices against the directory

### ◎ OBJECTIVE

A NAC server (PacketFence, or FreeRADIUS to learn the raw flow) that authenticates devices via 802.1X against FreeIPA, placing a known laptop on the LAN VLAN and quarantining an unknown device into an isolation VLAN with portal-only access.

```
# PacketFence: an all-in-one NAC (RADIUS + captive portal + isolation network).
# Easiest path: import the official ZEN appliance VM and run its configurator
# (https://packetfence.org). It provides RADIUS, role/VLAN assignment, isolation.
#
# To learn the 802.1X flow from first principles, FreeRADIUS is the minimal core:
sudo apt install freeradius -y
```

```
# FreeRADIUS backed by the Module-1 directory, trusting the switch as a client
# /etc/freeradius/3.0/clients.conf
client switch01 { ipaddr = 10.10.10.2; secret = <shared-secret> }
# mods-available/ldap -> point the server at FreeIPA so logins resolve to users

# authenticator (switch) -- the port stays closed until RADIUS says yes
interface GigabitEthernet0/5
 authentication port-control auto          ! enforce 802.1X here
 dot1x pae authenticator
 mab                                       ! fall back to MAC-auth for IoT/printers
```

The strongest posture reuses the PKI you already built: EAP-TLS, where the device proves itself with a certificate signed by your Module 10 CA — no password to phish or spray.

```
# supplicant (the client) -- EAP-TLS with the device cert from your Module-10 CA
network={
    key_mgmt=IEEE8021X
    eap=TLS
    identity="lan-client.lab.local"
    ca_cert="/etc/cert/root.crt"
    client_cert="/etc/cert/client.crt"
    private_key="/etc/cert/client.key"
}
# RADIUS reply attributes that tell the switch which VLAN to drop the device into
:
Tunnel-Type = VLAN
Tunnel-Medium-Type = IEEE-802
Tunnel-Private-Group-Id = "10"           # authenticated known device -> LAN VLAN 10
                                         # unknown / failed -> isolation VLAN (
                                         portal only)
```

```
# test the supplicant on a wired port and watch the EAP exchange succeed
sudo wpa_supplicant -i eth0 -D wired -c /etc/wpa_supplicant/8021x.conf -d
```

### RED TEAM — ATTACK

**Walk a rogue device in, then try to cheat MAB.** An unknown laptop with no cert and no credentials is dropped into the isolation VLAN — it can reach the remediation portal and nothing else. Now attack the weak fallback: clone an allowed printer's MAC to defeat MAB.

```
# spoof the printer's MAC to bypass MAB (the lesson, not a recommendation)
sudo ip link set eth0 address aa:bb:cc:11:22:33 # an allowed device's MAC
# against a MAB-only port this WORKS -- MAB trusts a forgeable identifier.
# against an EAP-TLS port it FAILS -- no valid certificate, no access.
```

### VERIFY IT WORKS

Three outcomes prove admission control: the known device with a valid certificate authenticates and lands on VLAN 10 with full LAN reach; the unknown device is quarantined to the isolation VLAN, able to reach only the portal; and the MAC-spoofed device gets in *only* where you weakly allowed MAB, but is blocked on the 802.1X-enforced ports. You now control *who joins*, not merely what flows once they are on.

### WATCH OUT

**NAC bites the careless.** It is operationally heavy — every device must be onboarded, and printers, IoT and legacy kit will break it, which is why MAB exists and why it is a *managed risk*, never a security control on its own. A misconfigured NAC can lock out an entire office, so keep a **break-glass** port the way Module 1 kept a break-glass account, and roll out in *monitor / audit* mode first — logging who *would* be denied — before you enforce. Same tune-before-blocking discipline as every other control in this lab.

**Pipeline tie-in.** 802.1X with EAP-TLS leans on the PKI (M10) and directory (M1); the isolation VLAN reuses your segmentation (M3); and “a new, unrecognised device appeared on the network” is itself a detection signal you forward to the SIEM (M8).

## MODULE 33

# SOC Skills Checklist (extends Module 21)

Tick a row only when you have done it and the Verify box passed. Add these to the Volume I mastery checklist.

| ✓                        | Skill                                                               | Module    |
|--------------------------|---------------------------------------------------------------------|-----------|
| <input type="checkbox"/> | Run an alert through the full lifecycle as a tracked case (TheHive) | SOC ops   |
| <input type="checkbox"/> | Apply a severity/priority matrix; write a triage runbook            | SOC ops   |
| <input type="checkbox"/> | Explain MTTD / MTTR / dwell time and why FP-rate matters            | SOC ops   |
| <input type="checkbox"/> | Stand up MISP; ingest a feed; enrich an indicator                   | CTI       |
| <input type="checkbox"/> | Explain the Pyramid of Pain; choose IOC vs TTP detection            | CTI       |
| <input type="checkbox"/> | Push intel into the SIEM as a watchlist that fires                  | CTI       |
| <input type="checkbox"/> | Investigate with KQL/SPL/Elastic: filter, aggregate, pivot          | Triage    |
| <input type="checkbox"/> | Find a brute-force <i>success</i> hidden among failures             | Triage    |
| <input type="checkbox"/> | Reconstruct an attack timeline across multiple sources              | Triage    |
| <input type="checkbox"/> | Automate enrichment + reversible containment (SOAR)                 | SOAR      |
| <input type="checkbox"/> | Encode a human-judgement branch + a protect-list guardrail          | SOAR      |
| <input type="checkbox"/> | Query the fleet live with osquery; ship a query pack                | EDR       |
| <input type="checkbox"/> | Hunt + respond across endpoints with Velociraptor                   | EDR       |
| <input type="checkbox"/> | Deploy Zeek; read conn/dns/http/ssl logs                            | NSM       |
| <input type="checkbox"/> | Detect beaconing by behaviour (RITA / conn.log)                     | NSM       |
| <input type="checkbox"/> | Detect DNS tunnelling from dns.log metadata                         | NSM       |
| <input type="checkbox"/> | Plant canary tokens, a honeypot, and a honey account                | Deception |
| <input type="checkbox"/> | Explain why deception is near-zero false positive                   | Deception |
| <input type="checkbox"/> | Run a hypothesis-driven hunt (stack counting / rarity)              | Hunting   |
| <input type="checkbox"/> | Convert a hunt finding into a portable Sigma detection              | Hunting   |
| <input type="checkbox"/> | Configure SPF, DKIM and DMARC for a domain                          | Email     |
| <input type="checkbox"/> | Investigate a phishing header (alignment, Received chain)           | Email     |
| <input type="checkbox"/> | Spoof an email and watch DMARC reject it                            | Email     |
| <input type="checkbox"/> | Build an isolated sandbox with INetSim + capture                    | Sandbox   |
| <input type="checkbox"/> | Detonate a specimen and extract behavioural IOCs                    | Sandbox   |
| <input type="checkbox"/> | Run an automated sandbox (CAPE); map to ATT&CK                      | Sandbox   |
| <input type="checkbox"/> | Deploy 802.1X NAC on the directory (PacketFence/FreeRADIUS)         | NAC       |
| <input type="checkbox"/> | Assign VLANs dynamically; quarantine an unknown device              | NAC       |
| <input type="checkbox"/> | Show why MAB is spoofable and EAP-TLS is not                        | NAC       |

**Where this points next.** This SOC track maps onto the Blue Team Level 1 (BTL1) and CompTIA CySA+ certifications, and prepares you for the blue-team paths on TryHackMe and LetsDefend. Paired with Volume I and the offensive volume (Part B, next), it is a portfolio that says “I can build it, attack it, *and* run the operations that defend it.”

*Part B — Offensive Security* continues the story from the attacker’s chair: pentest methodology and rules of engagement, OSINT and recon, web exploitation on the Juice Shop you already

provisioned, the full Active Directory kill chain against your own directory, privilege escalation, lateral movement and pivoting, command-and-control in the lab, and the report that is the real deliverable of an engagement.